

Contents/language:

1. Data types
2. Operators
3. Basic functions
4. Control flow
5. Function declarations/usage
6. Libraries

Language:

1. Data types

Variables inside the snake language can take values that are of type Int (integer), String and Array that can contain both integers and strings, as well as variables that are evaluated the moment they are used (when they are stored inside the Array as: [variable]). The code starts with the first line of code that the user writes and ends with the word end.

2. Operators

`+`: can be used to add two integers, concatenate two strings or arrays.

`-`: subtract an integer from an integer

`*`: multiply two integers or pop the head of the list aka the first element of the array (`array*num` - pops the head for any number used)

`/`: divide two integers (`a DIV b`) or get an array element (`array/element`)

`var <- expr`: assigns the value of the expression to the variable

`functdefinition [param] -> statement`: defines function

`expr1=expr2`: if `expr1 = expr2` then True else False

`expr1>expr2`: if `expr1 > expr2` then True else False

`expr1<expr2`: if `expr1 < expr2` then True else False

`->var`: frees the variable `var` from the program's memory

3. Basic functions

`print expr` - prints the expression

`read variable` - reads an input value from the user and assigns it to the variable

`return expr`: returns the expression `expr`

`random expr`: random number `< expr`

`import Library`: imports library

4. Control flow

If statements:

if logical expression **then** block of code **else** block of code **endif**

Example:

`read a`

`if a = 1 then`

`print 'a = 1' else`

```
print 'a != 1' endif  
end
```

If statements always need to include the else part and always need to end with the word endif.

5. Function declarations/usage

Funcdefinition [param] -> statement: defines function that takes [param] array as its parameters (functions need to start with a capital letters and variables with small letters)

do Function [param]: executes the function taking [param] as its parameters and if it includes a return statement saves the return value using the same name as the function.

var<-Function: if the Function returned a value, it can be assigned to a variable in this way.

Functions can be used recursively and even better: the recursion I have implemented is pretty effective, you can test it yourself with 1000000 recursions for example:

```
Loop [a] -> if a = 1000000 then print 'end' else print a a<-a + 1 do Loop[a]  
endif  
do Loop[1]  
end
```

Those 1000000 recursive calls don't take up too much ram and take less than a minute to be completed.

6. Libraries

For now StdLib, Math and Arrays:

StdLib has like some useful functions overall - the name was certainly not inspired by c

Math has some math function like Not, Xor, Factorial, Fibonacci etc

Arrays has some array related functions so you can remove certain elements from the array, change values inside the array etc. It is very useful as arrays are normally immutable.

Functions imported using the import function need to be addressed using the name of the library and the function name:

Ex addressing the Factorial function: do Math.Factorial [number]